

Generate and validate Terraform code using Kiro CLI's code generation capabilities and MCP Terraform server to ensure deployment-ready infrastructure configurations.

Related MCP Server:

- [AWS Terraform MCP Server](#): Terraform on AWS best practices, infrastructure as code patterns, and security compliance with Checkov.

After we have obtained the diagram, the next step is to try deploying this solution. Here, we will use Kiro CLI's code generation to write Terraform code and use MCP `terraform-mcp-server` to validate that code to ensure correct values by specifying additional information as shown in the following example:

1. Start by writing Terraform code

```
1 I want to test this architecture out , create a terraform code in folder terraform with the following information , assume sensible defaults
2 Project: "mywebapp"
3 Environment: "prod"
4 Region: "us-east-1"
5
6 Networking:
7 VPC CIDR: "10.0.0.0/16"
8 Public subnets: "10.0.1.0/24", "10.0.2.0/24"
9 Private subnets: "10.0.3.0/24", "10.0.4.0/24"
10 AZs: "us-east-1a", "us-east-1b"
11
12 Application:
13 Container image: "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest"
14 App port: 3000
15 ECS vCPU: 2, Memory: 256MB per task
16 Desired tasks: 4
17
18 Database:
19 Engine: MySQL 8.0
20 Instance: db.r6g.large
21 -storage: 50 GB
22 DB name: "webapp_db"
23 Master user: "admin"
24
25 Domain:
26 Domain: "myapp.example.com"
27 Health check: "/health"
28 S3 bucket: "myapp-static-assets"
```

The result is a complete Terraform file in the `terraform` folder with all necessary components that can be further edited and customized. You will notice that Kiro does not use `terraform-mcp-server` to generate terraform code.

```
> Created Terraform configuration in the terraform folder with your exact specifications:

Key Configuration:
• **ECS:** 2 vCPU, 256MB memory, 4 tasks (as requested)
• **VPC:** 10.0.0.0/16 with specified subnet CIDRs
• **Database:** MySQL 8.0 on db.r6g.large with 50GB storage
• **Container:** Your ECR image URI
• **Health Check:** /health endpoint
• **S3 Bucket:** myapp-static-assets

Files Created:
• terraform/main.tf - Complete infrastructure
• terraform/outputs.tf - Key output values

To Deploy:
bash
cd terraform
terraform init
terraform plan
terraform apply

Note: The S3 bucket name must be globally unique, so you may need to modify "myapp-static-assets" if it's already taken.
> █
```

2. Use MCP Terraform to validate the code before actual deployment (awslabsterraform_mcp_server)

```
1 I have terraform code at terraform folder , use terraform mcp server to validate the code.
2 You can run `terraform init` before start validation.
3 If you found error, please fix it for me.
```

Terraform MCP will perform validation and is able to fix the error (if found).

```
✖ Using tool: ExecuteTerraformCommand (trusted) from mcp server awslabs.terraform-mcp-server
:
• Running ExecuteTerraformCommand with the param:
: {
:   "command": "validate",
:   "working_directory": "/Workshop/mcp/terraform"
: }
:
:
• Completed in 3.393s

> ✅ Fixed and Validated Successfully!

Issue Found: cache_behavior block type was not supported in CloudFront distribution
Fix Applied: Changed cache_behavior to ordered_cache_behavior
Result: Terraform configuration is now valid and ready for deployment

The Terraform code in the terraform folder is now syntactically correct and passes validation.
```

Summary

You have successfully generated comprehensive Terraform code from your architecture diagram and validated it using MCP Terraform server. This demonstrates how Kiro CLI can accelerate Infrastructure as Code development by automatically creating deployment-ready configurations and identifying potential issues before deployment.